

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024
Reporte #9 - Semana 10 – Libro C - Capítulo 6
Estructuras

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 23/09/2024

Bibliografía:

B. W. Kernighan and D. M. Ritchie, *The C programming language*, 2nd ed. Pearson, 2002, pp. 114–134.

Abstract—Structures in C enable grouping variables of different types under a single name, simplifying data organization in complex programs. Key concepts include structure declaration, member access, and initialization. Passing structures to functions and using pointers to structures is explored for efficient memory management. Arrays of structures and self-referential structures, such as binary trees for dynamic data handling, are covered. Additionally, typedef is introduced for creating type aliases, while unions allow storing different data types in the same memory space, and bit-fields offer compact storage of information. Practical examples demonstrate the versatility and efficiency of structures in C programming.

I. Estructuras

Las estructuras en C permiten agrupar múltiples variables de distintos tipos bajo un mismo nombre, lo que facilita la organización y manejo de datos complejos en programas. Esto es especialmente útil en grandes programas, ya que las variables relacionadas se pueden tratar como una sola unidad. Un ejemplo clásico de uso de estructuras sería un registro de nómina que incluye atributos como nombre, dirección y salario de un empleado. En gráficos, una estructura puede representar un punto (con coordenadas x e y) o un rectángulo (con dos puntos opuestos).

II. Conceptos básicos de estructuras

Una estructura se declara usando la palabra clave struct, seguida por una lista de variables (miembros) entre llaves. Estas estructuras permiten agrupar datos y acceder a sus miembros usando el operador .. Por ejemplo:

```
struct point {  
    int x;  
    int y;  
};
```

```
struct point pt;
pt.x = 10;
pt.y = 5;
```

Esto permite definir tipos de datos personalizados. Las estructuras también pueden inicializarse directamente:

```
struct point maxpt = {320, 200};
```

Además, las estructuras pueden contener otras estructuras, permitiendo la creación de estructuras anidadas.

III. Estructuras y funciones

Las operaciones permitidas con estructuras incluyen copiar, asignar, acceder a miembros, y pasar estructuras a funciones. Las estructuras no pueden compararse directamente. Un ejemplo común es la función `makepoint`, que retorna una estructura `point`:

```
struct point makepoint(int x, int y) {
    struct point temp;
    temp.x = x;
    temp.y = y;
    return temp;
}
```

Otra función útil es `ptinrect`, que determina si un punto está dentro de un rectángulo dado:

```
int ptinrect(struct point p, struct rect r) {
    return p.x >= r.pt1.x && p.x < r.pt2.x && p.y >= r.pt1.y && p.y < r.pt2.y;
}
```

IV. Punteros a estructuras

En lugar de copiar estructuras grandes, es más eficiente pasar un puntero a la estructura. Un puntero a estructura se declara de la siguiente forma:

```
struct point *pp;
```

El operador `->` permite acceder a los miembros de una estructura a través de un puntero:

```
pp->x = 10;
```

Este operador es equivalente a `(*pp).x` pero más conciso y fácil de leer.

V. Arreglos de estructuras

Es posible declarar arreglos de estructuras. Por ejemplo, para contar las palabras clave en un programa C, se puede usar un arreglo de estructuras:

```
struct key {
    char *word;
    int count;
} keytab[] = {
    {"auto", 0}, {"break", 0}, {"case", 0}, {"char", 0}
};
```

Este arreglo almacena las palabras clave y un contador de sus apariciones. Un programa típico usaría este arreglo para contar y mostrar las palabras clave encontradas en el código.

VI. Punteros a estructuras en arreglos

Para evitar el uso de índices en un arreglo de estructuras, se puede usar punteros. Al hacerlo, las funciones que operan sobre estructuras pueden ser más eficientes y concisas. Por ejemplo, el siguiente código usa un puntero en la función `binsearch` para buscar palabras clave en una tabla:

```
struct key *binsearch(char *word, struct key *tab, int n) {
    struct key *low = &tab[0];
    struct key *high = &tab[n];
    while (low < high) {
        struct key *mid = low + (high - low) / 2;
        int cond = strcmp(word, mid->word);
        if (cond < 0) high = mid;
        else if (cond > 0) low = mid + 1;
        else return mid;
    }
    return NULL;
}
```

VII. Estructuras autorreferenciales

Las estructuras pueden referirse a sí mismas a través de punteros, lo que permite construir estructuras más complejas como árboles binarios. Cada nodo del árbol contiene un puntero a la izquierda y derecha, así como el valor almacenado:

```
struct tnode {
    char *word;
    int count;
    struct tnode *left;
    struct tnode *right;
};
```

El uso de estas estructuras autorreferenciales permite implementar eficientemente árboles binarios de búsqueda.

VIII. Tipos definidos con typedef

El comando `typedef` permite definir nuevos nombres para tipos existentes, facilitando la lectura del código. Por ejemplo:

```
typedef struct tnode *Treeptr;
```

Esto crea un alias `Treeptr` para `struct tnode *`. `typedef` también es útil para mejorar la portabilidad del código al definir tipos que pueden variar entre plataformas.

IX. Uniones

Las uniones son similares a las estructuras, pero permiten almacenar diferentes tipos de datos en el mismo espacio de memoria. Solo un tipo puede estar activo a

la vez, y el programador debe controlar cuál tipo está almacenado. Las uniones se declaran de la siguiente manera:

```
union u_tag {  
    int ival;  
    float fval;  
    char *sval;  
};
```

En este ejemplo, la variable u puede almacenar un entero, un flotante o un puntero a una cadena, pero solo uno de estos a la vez.

X. Campos de bits

Los campos de bits permiten ahorrar espacio al almacenar variables que solo necesitan unos pocos bits. Se usan comúnmente para almacenar banderas o datos compactos. Un campo de bits se define dentro de una estructura especificando el número de bits:

```
struct {  
    unsigned int is_keyword : 1;  
    unsigned int is_extern : 1;  
    unsigned int is_static : 1;  
} flags;
```

En este ejemplo, se definen tres campos de un bit que pueden ser usados para almacenar banderas.